

Demo Preparation Guide

Confidential — Internal Use Only

v1.0 | April 2026

Table of Contents

1. Demo Overview
2. Pre-Demo Checklist
3. Demo Narrative Script
4. Demo Flow (Step-by-Step)
5. Key Talking Points
6. Common Objections & Responses
7. Competitive Positioning
8. Technical FAQ
9. Follow-Up Template

1. Demo Overview

Purpose

This 30-minute demo showcases OpsLens AI's ability to detect, diagnose, and intelligently route critical incidents in real-time. The demo should demonstrate our AI-native approach to incident response and position OpsLens as the modern alternative to traditional incident management tools.

Demo Duration: 30 Minutes

Component	Duration
Opening Hook & Setup	2 min
Dashboard & Incident Feed	5 min
Root Cause Analysis	10 min
Routing & RRT Brief	8 min
Q&A & Pricing	5 min

Key Outcomes

- Audience sees OpsLens detect and diagnose a P1 incident in under 60 seconds
- Demonstrate AI-powered root cause analysis with concrete evidence (Jira, GitHub)
- Show intelligent routing based on service ownership and past incident patterns
- Illustrate how OpsLens reduces MTTR and alert fatigue
- Leave them wanting to pilot the product

Target Audiences & Their Priorities

Audience	Key Pain Point	What They Want to See
Engineering Lead	Alert fatigue, slow MTTR	Real-time diagnosis, context, reduced noise
CTO/VP Engineering	Team productivity, reliability	ROI, integration with existing tools, scalability
Investor/Executive	Market opportunity, defensibility	AI advantage, business model, market size

2. Pre-Demo Checklist

Complete 30 minutes before the demo:

Environment Setup

- SSH into production OpsLens instance (or staging if live data unavailable)
- Verify database connections are stable (PostgreSQL, Qdrant)
- Check all API services are running: FastAPI, Celery workers, background tasks
- Confirm Next.js frontend is running and accessible at correct URL

Test Data Preparation

- Pre-load sample incident data: PaymentService P1 with live logs
- Verify Jira integration is configured and ticket links resolve
- Confirm GitHub PR enrichment shows the commit that caused the incident
- Test Slack integration: verify routing rules exist for #payment-team and #openai-alerts
- Generate a sample RRT brief to ensure template rendering works

Backup & Fallback Plans

- Have a local PDF of incident logs as fallback if live data unavailable
- Pre-record a 2-min video walkthrough of the key demo flow
- Keep a phone nearby to text technical team in case of issues
- Have backup Wi-Fi hotspot ready if office network fails

Browser & Workspace Setup

- Open OpsLens dashboard in primary monitor (full screen, 1920x1080 or higher)
- Open speaker notes in secondary monitor or tablet
- Have Jira and GitHub open in background tabs for quick reference
- Clear browser cache and history to ensure fresh loads
- Disable Slack notifications during demo to avoid interruptions

Final Verification (5 min before)

- Test mouse and keyboard responsiveness
- Verify audio/video conferencing tool if remote demo
- Do a quick run-through of the first 3 steps of the demo flow
- Confirm the PaymentService incident is visible on the dashboard
- Check that your phone is on silent

3. Demo Narrative Script

The Narrative Arc: A P1 Incident at 3am

OPENING HOOK (2 min)

"It's 3am. Your payment service just went down. Your team is asleep. Your CEO is about to call. What happens in the next 60 seconds?"

Traditionally: You wait for PagerDuty to fire, manually dig through logs for 20 minutes, guess where the problem is, ask someone on Slack, lose 2 hours of revenue.

With OpsLens: Let me show you what happens instead."

ACT 1: THE DASHBOARD (5 min)

[SHOW the OpsLens dashboard with live incident feed]

"This is the OpsLens dashboard. Notice three things:"

1. Real-time incident feed: We automatically ingest logs from your infrastructure. No manual setup.
2. Severity classification: This PaymentService incident is a P1 - we know it's critical because the error rate spiked 400% in 30 seconds.
3. AI-powered context: We've already run diagnostics. See the red icon? That means we've identified the likely root cause."

ACT 2: ROOT CAUSE ANALYSIS (10 min)

[CLICK into the PaymentService incident]

"Now we're in the incident details page. Watch what OpsLens has already done:"

1. AI Diagnosis: We parsed 50,000 log lines in 2 seconds and identified the root cause: "ConnectionPoolException in PaymentProcessor:429"
2. Evidence from Jira: [CLICK the Jira link] Here's the open ticket - Database pool exhaustion. Our team triaged this last week but didn't deploy the fix.
3. Evidence from GitHub: [CLICK the GitHub PR link] Here's the exact commit that would have fixed it. It's waiting on code review. We can see it was authored 3 days ago.
4. Related Alerts: We've linked this to 12 other incidents with the same root cause from the past 6 months. No surprises here - this is a known issue."

ACT 3: INTELLIGENT ROUTING (5 min)

[NAVIGATE to Routing Rules]

"Here's where OpsLens saves your night: routing rules."

[SHOW the rule: If service=payment-service AND severity=P1 THEN route to #payment-team]

We didn't guess. We looked at your Jira project structure and Slack workspace. We know who owns payment services. And we know who's on-call. Instead of alerting 20 people on #incidents, we route directly to the 3 people who can actually fix it."

[SHOW second rule: If error_type contains "AI" THEN also route to #openai-alerts]

You can also create compound rules. This one says: if it's an AI inference failure, also ping the AI team. No alerts are lost. But no one gets spam either."

ACT 4: RRT BRIEF (5 min)

[NAVIGATE to the auto-generated RRT brief]

"Finally, the thing on-call engineers actually need: a 30-second brief."

[SHOW the RRT brief output]

ROOT CAUSE: Database connection pool exhausted. 427 active connections vs. 425 limit.

BLAST RADIUS: Payment processing API, 98% error rate, ~\$15K revenue/min impacted.

QUICK FIX: Increase pool size from 425 to 500 in config (10 seconds). Or deploy PR #4821.

OWNING TEAM: payment-team. Contact: Priya (on-call), backup: Marco.

CONTEXT: This is the 3rd time this month. PR #4821 would fix permanently.

This is generated in real-time. No manual brief. The on-call engineer reads this in 30 seconds, knows exactly what to do."

CLOSE (3 min)

"That's OpsLens in action. One incident. 60 seconds. Maximum context. Minimum noise."

Questions before we dive into pricing and technical questions?"

4. Demo Flow (Step-by-Step)

Exact UI interactions with talking points:

Step 1: Land on Dashboard

Action: Action: Browser is already open to OpsLens dashboard

Say: Say: 'This is OpsLens. All your incidents in one place. Real-time, AI-powered.'

Click: Click: Nothing yet - let them look for 5 seconds

Step 2: Show Incident Feed

Action: Action: Scroll to show PaymentService P1 incident is visible

Say: Say: 'Notice this PaymentService incident at the top. It's marked P1 - critical severity. It appeared 90 seconds ago.'

Click: Click: Hover over the incident to show preview tooltip

Step 3: Click into Incident

Action: Action: Click the PaymentService P1 incident row

Say: Say: 'Let me click into this to show you what we've already figured out.'

Click: Click: Main incident details page loads

Step 4: Show AI Analysis Panel

Action: Action: Incident details page is now visible

Say: Say: 'Here's the AI diagnosis. In 2 seconds, we parsed 50,000 log lines and identified this: ConnectionPoolException.'

Click: Click: Point to the diagnosis card (don't click, just highlight)

Step 5: Show Jira Enrichment

Action: Action: Scroll down to show linked Jira ticket

Say: Say: 'And here's the related Jira ticket. This is a known issue that was triaged last week.'

Click: Click: Click the Jira link to open the ticket in new tab (optional)

Step 6: Show GitHub Enrichment

Action: Action: Scroll to show linked GitHub PR

Say: Say: 'Here's the exact commit that would fix this. It's awaiting code review. The fix is 3 lines.'

Click: Click: Click GitHub link (optional) or just highlight the PR number

Step 7: Navigate to Routing Rules

Action: Action: Click 'Routing Rules' in left navigation

Say: Say: 'Now let me show you how OpsLens routes this incident.'

Click: Click: Routing Rules page loads

Step 8: Show Routing Rule for Payment Team

Action: Action: Find the rule 'service=payment-service AND severity=P1 -> #payment-team'

Say: Say: 'This rule says: if it's a critical payment incident, route to the payment team. We configured this based on your Jira and Slack.'

Click: Click: Hover to show rule preview

Step 9: Navigate to Incident Simulator

Action: Action: Click 'Simulate Alert' in navigation

Say: Say: 'Let me show you how easy it is to test new rules.'

Click: Click: Simulator page loads

Step 10: Run a Simulation

Action: Action: Fill in form: service='payment-service', error='ConnectionPoolException'

Say: Say: 'I'll run a simulation with the same error. Watch where it routes.'

Click: Click: Run Simulation button

Step 11: Show Routing Result

Action: Action: Simulator shows result: 'Routed to: #payment-team'

Say: Say: 'Perfect. It matched our rule and routed to the right team.'

Click: Click: No click needed

Step 12: Show RRT Brief

Action: Action: Click on the incident again to show auto-generated RRT brief

Say: Say: 'And here's what that on-call engineer sees. 30 seconds. All context. No guessing.'

Click: Click: View RRT Brief button

5. Key Talking Points

For Engineering Leaders

- MTTR Reduction: OpsLens typically reduces mean time to resolution by 2.4 hours per incident
- Alert Fatigue: 60-70% reduction in alert noise through intelligent filtering and deduplication
- Context is King: Engineers spend less time digging through logs, more time fixing
- Integration-First: Works with your existing tools (Jira, GitHub, Slack) - not a replacement
- Real AI: We use semantic search + RAG to surface actual context, not just keyword matching

For CTOs / VP Engineering

- Business Impact: Average downtime costs \$1.5M per incident for mid-market companies
- ROI: If OpsLens prevents just 2-3 serious incidents per year, it pays for itself 10x over
- Team Capacity: Your engineers stop becoming 24/7 on-call. They sleep. They ship features.
- Scalability: Works for 5-engineer teams and 500-engineer organizations. Same product, same pricing model
- Defensibility: AI is hard to replicate. Our models are trained on thousands of incident patterns

For Investors / Executives

- Market Size: \$4.2B TAM in incident management. Growing 25% annually as cloud-native adoption increases
- AI-Native Wedge: We're not adding AI to an old tool. OpsLens is built on AI from the ground up
- Usage-Based Economics: Customers pay for what they use. More incidents = more value = more revenue. Perfect alignment
- Differentiation: Datadog and PagerDuty are bolt-on solutions. We're purpose-built for incident diagnosis
- Team & Traction: [Your team background]. Already in private beta with [leading companies]

6. Common Objections & Responses

We already have PagerDuty / Datadog. Why do we need OpsLens?

PagerDuty and Datadog are great, but they're alert routing and monitoring tools, not diagnosis tools. OpsLens doesn't replace them - it plugs into your existing stack and adds AI-powered diagnosis. Think of it as the brain on top of your monitoring. You still get alerts from Datadog. OpsLens just makes those alerts actually useful by adding root cause analysis and intelligent routing.

We're too small for this. We only have 8 engineers.

Actually, smaller teams benefit the most. When your team is small and distributed, every incident eats hours of time. One person has to dig, one person has to be woken up, one person has to brief the CEO. OpsLens is designed for exactly this: fast diagnosis, right person gets the alert, zero toil. And our pricing scales with you - you only pay for actual incidents.

How does the AI actually help? Isn't it just keyword matching?

Nope. We use semantic search (embeddings) + retrieval-augmented generation (RAG). We don't just find the word 'timeout' in your logs. We understand what caused timeouts in the past, find related Jira tickets and GitHub commits, and synthesize that into a diagnosis. It's closer to asking a senior engineer who has context on your system.

What about data privacy? We can't send our logs to the cloud.

Totally fair. That's why we offer self-hosted deployment. OpsLens runs in your VPC. Your logs never leave your infrastructure. We only store deduplicated, anonymized incident patterns. Some customers do use our cloud for speed, and we sign BAAs and comply with SOC2, GDPR, HIPAA as needed.

How does pricing scale? We're worried about bill shock.

We charge per incident analyzed, not per user or per GB ingested. If you analyze 1,000 incidents this month, you pay for 1,000. Next month, if you have fewer incidents, your bill is lower. No seat licenses, no overages. Most customers see \$500-2K/month depending on incident volume. For a team of 10-50 engineers, that's typically break-even with just one prevented incident.

How long does setup take? We're in the middle of a big project.

Integration is typically 4-6 hours. We handle: Jira connection, GitHub integration, Slack routing config, log ingestion setup. Most of that is us talking to your team, not you building anything. You can be live in a day.

7. Competitive Positioning

OpsLens vs. The Alternatives

Capability	OpsLens	PagerDuty	Datadog	Manual Process
Real-time Alert Routing	Yes	Yes	No	N/A
Root Cause Analysis	AI-powered	Manual	Basic	Manual
Log Enrichment (Jira/GH)	Yes	Limited	No	Manual
Auto-generated Brief	Yes	No	No	Manual
Cost per Incident	\$0.50-2	Per user	Per GB	Per hour (lost)
Setup Time	4-6 hours	1-2 days	1-2 weeks	N/A
AI-Native	Yes	No	No	N/A

OpsLens Positioning Statement

"OpsLens is the first AI-native incident diagnosis platform. Unlike alert routing tools (PagerDuty) or monitoring platforms (Datadog), OpsLens is built specifically to answer the question every engineer asks at 3am: What broke and how do I fix it? In 60 seconds. With full context."

8. Technical FAQ

For Engineering Leads and CTOs

How do logs get ingested?

Logs stream in via: (1) Syslog/TCP, (2) Cloud provider SDKs (AWS CloudWatch, GCP Cloud Logging, Azure Monitor), (3) log forwarding agents (Fluentd, Logstash, Vector), (4) API. No agent required on application servers - we ingest from your log aggregation layer.

What AI models do you use?

We use a combination: (1) Embedding models (OpenAI text-embedding-3-small) for semantic search, (2) a fine-tuned LLM for incident diagnosis, (3) rule-based pattern matching for known incident types. All models run on-premise if you self-host. We don't share your data with OpenAI.

How does RAG enrichment work?

RAG = Retrieval-Augmented Generation. We vectorize your logs and store embeddings in Qdrant (our vector DB). When an incident occurs, we: (1) Search Qdrant for similar past incidents, (2) Fetch context from Jira/GitHub, (3) Pass that context + current logs to our LLM for diagnosis. Result: context-aware, grounded diagnoses.

What are the latency SLAs?

Diagnosis latency: < 5 seconds (p99) from log ingestion to diagnosis. Routing latency: < 2 seconds. RRT brief generation: < 3 seconds. End-to-end from incident to Slack notification: < 10 seconds. Cloud deployment. Self-hosted can be < 2 seconds depending on your hardware.

Where is data stored?

Cloud: AWS us-east-1 (can be configured). Backups encrypted at rest. Self-hosted: your VPC, your choice of region. All data encrypted in transit (TLS 1.3). Logs are deduplicated after 30 days and incident patterns are anonymized. Raw logs are not stored longer than 7 days unless you configure retention.

What are the prerequisites?

Minimum: (1) Log aggregation layer (CloudWatch, Datadog, Splunk, etc.), (2) Jira or GitHub integration, (3) Slack workspace. Optional: PagerDuty for on-call info. For self-hosted: Kubernetes cluster, PostgreSQL, minimum 4 CPUs + 8GB RAM.

How do you handle false positives?

Every diagnosis is labeled by our LLM with a confidence score (0-100). Engineers can provide feedback (yes/no/needs-more-context) which trains our models. After 50 diagnoses, models adapt to your specific patterns. False positive rate improves from 15% to < 5% over time.

Can we control what gets routed?

Yes, fully configurable. Routing rules are YAML-based and support AND/OR logic. You can route by: service, severity, error type, team ownership, time of day, frequency, etc. Rules are tested in simulation mode before going live.

9. Follow-Up Template

Email to send within 2 hours of demo:

Subject: OpsLens AI Demo - Quick Follow-Up + Next Steps

Hi [Name], Thanks for taking the time to see OpsLens in action today. I hope you got a feel for how it handles the 3am incident scenario we walked through. Here's what we covered: • OpsLens detects and diagnoses incidents in real-time using AI • Context from Jira, GitHub, Slack gets linked automatically • Routing rules ensure the right team gets the alert • RRT brief gives on-call engineers everything they need in 30 seconds You asked some great questions about [specific topic]. Here's a bit more detail: [INSERT ANSWER HERE] Next steps: 1. I'll send you a technical spec document (attached) for your engineering team to review 2. We can schedule a 30-min technical deep-dive with your [Platform / SRE] team if that's helpful 3. I'd love to run a 2-week free pilot. We'd integrate with your staging environment, ingest your logs, and see what OpsLens can diagnose on your actual incident patterns Would any of those work for you? [CTO] and I are flexible on timing. In the meantime, feel free to forward this to anyone on your team who should know about it. Happy to answer technical questions async on email or Slack. Looking forward to building with you. Best, [Your Name] [Title] OpsLens AI [Email] | [Phone] | [Website]

Follow-Up Checklist:

- Send email within 2 hours of demo
- Attach technical spec + pricing doc
- Answer any questions from demo within 24 hours
- Schedule follow-up call within 5 business days
- Send pilot agreement by end of week
- If no response in 5 days, send gentle reminder